

Lecture 22

Algorithmic Foundations of Multi-Agent Reinforcement Learning

COMP532 – Machine Learning and Bioinspired Optimisation

Meng Fang, University of Liverpool



Today's Questions

At the end of this lecture, you should be able to answer:

1. How do we move from game-theoretic concepts to concrete multi-agent RL algorithms?
2. What is an independent learner, and why is it attractive?
3. Why does non-stationarity make independent learning fundamentally difficult?
4. What additional structure is needed for joint-action and equilibrium-based learners?

Recap: From Static Equilibrium to Learning Dynamics

So far, we have built the following picture:

- ▶ Lecture 20: game theory gives us strategic solution concepts
- ▶ Lecture 21: repeated interaction creates learning dynamics

Today's step

We now ask how to design **algorithms** that allow agents to learn in multi-agent environments.

Central tension

We want scalable reinforcement learning methods, but the environment is no longer stationary.

What Makes Multi-Agent RL Algorithmically Hard?

Compared with single-agent RL, multi-agent RL introduces several new algorithmic difficulties:

- ▶ the transition and reward process depends on **other learners**
- ▶ each agent's update changes the learning problem of others
- ▶ exploration by one agent induces noise for another
- ▶ the relevant objective may be individual, joint, or equilibrium-based

Main challenge

How can an agent update its policy when the target it is learning against is itself changing?

A First Taxonomy of MARL Algorithms

A useful high-level division is:

1. **Independent learners**

- ▶ each agent runs RL as if others were part of the environment

2. **Joint-action learners**

- ▶ agents explicitly reason about the joint action

3. **Equilibrium-based learners**

- ▶ agents use game-theoretic solution concepts inside learning

4. **Gradient / policy search methods**

- ▶ agents directly optimise policies in interactive environments

Guiding idea

Different algorithm families make different assumptions about observability, structure, and equilibrium.

Independent Learners: The Simplest Idea

The most direct approach is:

Independent learning

Each agent treats all other agents as part of the environment and applies a standard RL algorithm locally.

For example, agent i may maintain its own $Q_i(s, a_i)$ and update it using:

$$Q_i(s, a_i) \leftarrow Q_i(s, a_i) + \alpha \left(r_i + \gamma \max_{a'_i} Q_i(s', a'_i) - Q_i(s, a_i) \right).$$

- ▶ simple
- ▶ scalable
- ▶ directly reuses single-agent RL machinery

Why Independent Learning is Attractive

Independent learners remain popular because they are:

- ▶ easy to implement
- ▶ computationally light
- ▶ compatible with standard RL infrastructure
- ▶ often surprisingly effective in practice

Engineering appeal

If single-agent RL works well, the first instinct is often to run one learner per agent.

But

This simplicity comes at a conceptual cost.

The Core Problem: Non-Stationarity

From the viewpoint of agent i , the environment includes the policies of the other agents:

$$P(s', r_i \mid s, a_i) \text{ depends on } \pi_{-i}.$$

If other agents are learning, then π_{-i} changes over time.

Consequence

The effective transition and reward process faced by agent i is **non-stationary**.

This breaks a key assumption behind many standard RL convergence arguments.

Why Non-Stationarity Breaks Standard Q-Learning Intuition

In standard Q-learning, the update target is assumed to be generated from a stationary environment.

In independent multi-agent learning:

- ▶ the reward distribution may shift
- ▶ the transition dynamics may shift
- ▶ the meaning of an action may shift as others adapt

Interpretation

The agent is trying to estimate a moving target.

A Simple Illustration of Moving Targets

Suppose agent 1 is learning whether action A or B is better.

But the return of each action depends on agent 2's current policy.

If agent 2 changes from mostly playing A to mostly playing B , then:

- ▶ the value of agent 1's actions changes
- ▶ old experience becomes less reliable
- ▶ the estimated Q-values may drift or oscillate

Lesson

Experience replay from the past may describe a different effective environment from the present one.

When Independent Learners Can Still Work

Despite the theoretical issue, independent learners can work reasonably well when:

- ▶ other agents change slowly
- ▶ interaction is weak or approximately stationary
- ▶ the environment is highly structured
- ▶ the task is cooperative and agents share informative signals

Balanced view

Independent learning is not always wrong. It is often a useful baseline, but it should not be mistaken for a principled solution in general.

Joint-Action Learners

A stronger approach is to model the value of **joint actions** explicitly.

For example, an agent may estimate:

$$Q_i(s, a_1, a_2, \dots, a_n).$$

This allows the learner to ask:

- ▶ what happens under different combinations of actions?
- ▶ how should I respond to specific opponent choices?
- ▶ what joint behaviour is desirable or stable?

Advantage

Joint-action learners represent strategic interaction directly.

The Price of Joint-Action Learning

Joint-action methods require more structure and information:

- ▶ the agent may need to observe the actions of others
- ▶ the value table grows with the size of the joint action space
- ▶ coordination becomes harder as the number of agents increases

Scalability issue

If each of n agents has m actions, the joint action space can scale like m^n .

This makes joint-action methods difficult in large systems.

Zero-Sum Settings are Special

In two-player zero-sum games, the structure is cleaner:

- ▶ one player's gain is the other's loss
- ▶ strategic interaction can be analysed through minimax reasoning
- ▶ the equilibrium concept is often more sharply defined

Why this matters

Many equilibrium-based learning methods are easier to formulate and analyse in zero-sum games than in general-sum games.

Minimax-Q Learning

In two-player zero-sum Markov games, one can generalise Q-learning using minimax reasoning.

Instead of a simple $\max$ over actions, the update target involves a minimax value:

$$V_i(s) = \max_{\pi_i} \min_{\pi_{-i}} \mathbb{E}_{a_i \sim \pi_i, a_{-i} \sim \pi_{-i}} [Q_i(s, a_i, a_{-i})] .$$

The update then uses the equilibrium value of the stage game induced by Q .

Interpretation

The learner plans against a strategically optimal adversary.

Why Minimax-Q is Conceptually Important

Minimax-Q shows how game-theoretic solution concepts can be embedded directly into RL updates.

It teaches us that:

- ▶ multi-agent value updates may require solving a game at each state
- ▶ equilibrium reasoning can replace simple maximisation
- ▶ the notion of optimality changes with the interaction regime

Key lesson

In multi-agent RL, the backup operator itself may need to be game-theoretic.

General-Sum Games are Harder

In general-sum games:

- ▶ agents may partly cooperate and partly compete
- ▶ multiple equilibria may exist
- ▶ equilibrium selection becomes a problem
- ▶ the relevant solution concept may be ambiguous

Why this matters

There may be no single canonical analogue of the single-agent Bellman optimality operator.

Nash-Q Learning

A natural general-sum extension is to replace the max or minimax backup by a Nash equilibrium computation.

At each state, the learners consider the stage game induced by the current Q -values and compute a Nash equilibrium.

The update target then depends on the equilibrium payoffs.

Idea

$$Q_i(s, \mathbf{a}) \leftarrow r_i + \gamma \cdot \text{EquilibriumValue}_i(s')$$

where the equilibrium value is derived from the stage game at the next state.

Challenge

This requires equilibrium computation inside learning, which can be expensive and unstable.

Why Equilibrium-Based Methods are Powerful but Heavy

Equilibrium-based learners are appealing because they:

- ▶ explicitly respect strategic structure
- ▶ connect RL to game-theoretic solution concepts
- ▶ can provide principled targets in some settings

But they also require:

- ▶ a solution concept to commit to
- ▶ equilibrium computation during training
- ▶ care with multiple equilibria and selection issues

Trade-off

More strategic sophistication usually means more computational and modelling burden.

Gradient-Based and Policy Search Methods

Another family of methods directly optimises policies, rather than explicitly solving value backups via equilibrium computation.

For agent i , a generic update takes the form

$$\theta_i \leftarrow \theta_i + \alpha \nabla_{\theta_i} J_i(\theta_i, \theta_{-i}),$$

where θ_i parameterises the policy ($\pi_{\theta_i}(a_i | s)$) of agent i and $J_i(\theta_i, \theta_{-i}) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_i^{(t)}]$. (J denotes the expected return induced by the policy.)

These methods may:

- ▶ update policies by gradient ascent on expected return
- ▶ adapt continuously rather than via discrete best responses
- ▶ induce non-trivial learning dynamics as multiple agents update simultaneously

Why they matter

Gradient-based learners provide a bridge between MARL algorithms and the dynamical systems viewpoint that we will study later.

A Unifying Comparison

Method family	Core idea	Needs others' actions?	Strategic structure	Main limitation
Independent learners	Treat others as environment	No / weakly	Implicit	Non-stationarity
Joint-action learners	Learn over joint actions	Yes	Explicit	Joint-space explosion
Minimax-Q	Solve zero-sum stage games	Typically yes	Strong	Mostly zero-sum
Nash-Q	Solve equilibrium at each state	Typically yes	Strong	Expensive / multiple equilibria
Gradient methods	Direct policy adaptation	Not always	Variable	Complex dynamics

A Deeper Lesson About Backup Operators

In single-agent RL, the Bellman optimality update uses:

$$\max_{a'} Q(s', a').$$

In multi-agent RL, this may be replaced by:

- ▶ no strategic correction (independent learners)
- ▶ explicit dependence on joint actions
- ▶ a minimax operator
- ▶ a Nash equilibrium operator

Core insight

The backup rule reflects the assumed structure of interaction.

What Should We Learn From This Taxonomy?

There is no universally best MARL algorithm.

The right method depends on:

- ▶ whether the setting is cooperative, competitive, or mixed
- ▶ whether others' actions are observable
- ▶ whether equilibrium structure is important
- ▶ whether scalability or strategic fidelity matters more

Practical principle

Algorithm choice in MARL is inseparable from assumptions about the game.

Why This Sets Up the Next Part of the Module

So far, we have seen:

- ▶ strategic solution concepts
- ▶ learning dynamics under repeated play
- ▶ algorithmic families for multi-agent RL

A natural next question is:

Big question

Can we analyse learning algorithms themselves as **dynamical systems**?

This leads naturally to evolutionary game theory and population dynamics.

Summary

1. Independent learners are simple and scalable, but suffer from non-stationarity.
2. Joint-action learners model strategic interaction more explicitly, but scale poorly.
3. Zero-sum settings admit minimax-style learning.
4. General-sum settings often require equilibrium-based reasoning.
5. The design of a MARL algorithm depends on how we model interaction and optimality.

Looking Ahead

Next lectures:

- ▶ Can we understand adaptation without assuming full rationality?
- ▶ What happens at the population level when better-performing strategies spread?
- ▶ How are learning algorithms connected to evolutionary dynamics?

Bridge to Lecture 23

We now move from **algorithmic MARL** to **evolutionary and dynamical perspectives on learning**.

Key Takeaways

- ▶ Multi-agent RL is harder than single-agent RL because the environment is strategic and non-stationary.
- ▶ Independent learning is a useful baseline, but not a general solution.
- ▶ Richer MARL methods explicitly model joint actions or equilibrium structure.
- ▶ The backup rule in MARL reflects assumptions about the underlying game.

Core message:

In multi-agent RL, learning rules are inseparable from assumptions about interaction.